Grade A Plus - Backend

Data Privacy, Security & Compliance

How the Grade A Plus backend stores school data, decides who may see it, and keeps it safe.

Second edition, 26 October 2026. This edition adds three things the first did not have: the laws and standards we work to, the encryption we use at every layer and who enforces it, and the measures that stand between the system and an attacker.

Contents

- 1. What the Backend Is
- 2. The Journey of One Paper
- 3. Everything the Backend Does
- 4. The Data We Hold
- 5. Who Can See What
- 6. Signing In Safely
- 7. How AI Is Used on Student Work
- 8. Payments
- 9. A Record of Who Did What
- 10. Keeping and Removing Data
- 11. Encryption, Layer by Layer
- 12. Defending the System
- 13. The Laws and Standards We Work To
- 14. The Companies We Work With
- 15. How We Check Our Own Work
- 16. At a Glance
- 17. Reporting a Security Concern
- Appendix: Where Each Statement Comes From

1. What the Backend Is

The backend is the part of Grade A Plus that users never see. The websites that teachers, students and school staff use are the front counter. The backend is everything behind it: the filing room, the marking desk, the accounts office and the security guard.

When a teacher opens a class or a student looks at a grade, the website asks the backend a question. The backend checks who is asking, decides what that person is allowed to see, and sends back only that. The websites hold no data of their own. Everything is kept, decided and recorded by the backend.

It has five standing jobs:

Job	What it means
Gatekeeper	Checks every single request and answers only with what that person is allowed to see
Record keeper	Holds accounts, classes, student work and the official record of every grade
Grading assistant	Reads student work with the help of AI and brings back a suggested grade for the teacher

Job	What it means
Cashier	Works with our payment provider for subscriptions, school licences and grading credits
Witness	Keeps a permanent record of who did what, and when

The next two sections show those jobs in action. The rest of the document explains how each is done safely.

2. The Journey of One Paper

The clearest way to see what the backend does is to follow one piece of student work from start to finish.

1. **The teacher sets the work.** The teacher writes an assignment, or asks the AI to help draft the questions and a marking guide. The backend saves it against that teacher's class.
2. **The work comes in.** The teacher uploads the students' papers, one at a time or a whole class at once. The backend checks each file is a genuine PDF of an acceptable size.
3. **The paper is read.** The backend turns each page into a picture and asks the AI to read the answers, including handwriting. It matches the paper to the right student using the class list.
4. **The answers are marked.** The backend sends the answers and the teacher's marking guide to the AI, which suggests a score and written feedback for each question.
5. **The marking is double-checked.** A second AI gives an independent opinion. Where the two disagree, or where a page was hard to read, the backend flags the paper so the teacher knows to look closely.
6. **The teacher reviews.** The suggested grade goes to the teacher, not the student. The teacher can accept it, change it or ask for it to be marked again. The backend keeps the AI's original suggestion beside any change.
7. **The teacher releases the grade.** Only then can the student see it. The backend sends the student an email to say their grade is ready, if the student has an email address.
8. **Everything is recorded.** Each of these steps leaves an entry in the permanent activity record, and each use of AI is counted against the teacher's or school's grading credits.

The heavy work in steps 3 to 5 happens in the background. The teacher can carry on using the site, and the backend reports progress as each paper is finished.

3. Everything the Backend Does

Marking is the heart of the service, but the backend does a good deal more. Here is the full picture, in plain terms.

Accounts and sign-in

- Creates accounts for teachers, school administrators and students, and confirms email addresses.
- Signs people in with a password or with Google, and signs them out.
- Handles forgotten passwords with short-lived emailed codes.
- Sends invitations when a school adds a teacher or a teacher adds a student.

Schools, classes and students

- Keeps each school's list of teachers, and each teacher's school terms, courses and class lists.

- Adds students to a class one by one or from a whole list, and records when a student joins, completes or leaves a course.
- Moves a teacher into or out of a school, and adjusts what they can reach when that happens.

Assignments

- Stores each assignment with its questions, instructions and marking guide.
- Helps a teacher draft an assignment through a conversation with the AI.
- Produces a clean, printable PDF of an assignment for the classroom.
- Can start marking automatically when an assignment’s due date arrives. It never releases grades automatically.

Marking and feedback

- Reads uploaded papers, marks them and double-checks the marking, as described in section 2.
- Works out each student’s final grade for a course from their marked work.
- Writes a short summary of how a student is progressing, for the teacher.
- Remembers marking it has already done for an identical answer for a few days, so the same answer is marked the same way and no credit is spent twice.

Insight for teachers and schools

- Builds each teacher’s dashboard: class averages, completion and which students may need help.
- Builds each school administrator’s dashboard: results across the school, by course and by teacher.
- Writes weekly summaries for courses and for the school.
- Answers a teacher’s or administrator’s questions about their own figures through an AI assistant.

Payments, licences and credits

- Runs individual teacher subscriptions and school-wide licences with a set number of teacher seats.
- Records payments made by card and payments a school makes by invoice.
- Keeps a running account of grading credits: what was bought, what was granted to each teacher and what each piece of AI work used.
- Gives credit back when a piece of AI work fails.
- Handles renewals, cancellations, refunds and payment disputes.

Messages

- Sends the emails the service depends on: confirmations, invitations, password codes, grade notices and billing notices.
- Respects each person’s notification choices.

Work done on a timetable

Some jobs run by themselves, every day or every few minutes, with nobody asking:

- Checking our billing records against the payment provider’s, to catch any mismatch.
- Retiring grading credits that have reached their end date.
- Sending reminders before a subscription renews or a licence runs out.

- Removing old entries from the activity record once their keeping time is over.
- Checking its own health and raising an alert to our engineers if something is wrong.

Keeping watch

- Writes every important action into the permanent activity record.
- Reports faults to our engineers, with personal details left out.
- Measures the AI's marking against papers marked by hand, so we know how accurate it is.

4. The Data We Hold

We collect as little as we can. The less we hold, the less there is to protect.

What we hold

- **Teachers and school staff:** name, email address, school, and an optional profile picture.
- **Students:** name, the classes they are in, their answers, their grades and the feedback on their work.
- **Payments:** reference numbers, amounts and dates.

A student does not need an email address. A teacher can add a student by name alone, and no email is ever sent to a student added that way.

What we never hold

- **Sensitive facts about students.** We have no place to store a date of birth, gender, race, religion, disability, home address, phone number or parent details. The system was built without them.
- **Card numbers.** They go straight to our payment provider and never reach us.
- **Readable passwords.** Passwords are scrambled before they are stored, so nobody can read them back, including us.
- **Copies of scanned papers.** When a teacher uploads a scanned paper, we read the answers from it and keep the answers as text. The file itself is held only for the minutes it takes to read it, then removed.

How it is stored

- The database is encrypted where it is stored, by our hosting company.
- Everything travelling between a user and the backend is encrypted.
- The keys that let a user sign in with Google get a second layer of encryption of their own, inside the backend.

Section 11 explains each of these in detail, names the standard used, and says who enforces it.

5. Who Can See What

There are four kinds of user. The backend decides what each one may see, and it decides again on every request.

Who	What the backend lets them see
Student	Their own courses and their own work. Grades and feedback appear only after the teacher releases them.

Who	What the backend lets them see
Teacher	Their own classes and the students in them. Nothing from another teacher’s classes.
School administrator	Their own school only: its staff, its students and school-wide results. Never another school.
Grade A Plus support team	A small number of our own staff, with the access needed to run the service and help schools.

- **No school data is shown without signing in.** The backend refuses any request for data that does not come from a signed-in user. (The only pages open to everyone are the sign-in doors themselves and the technical description of the backend’s own interface, which is standard practice and holds no data.)
- **The backend never trusts the screen.** A web page may hide a button, but the backend makes the real decision. If someone got around a screen, the backend would still refuse.
- **One school cannot reach another’s data.** Each answer is built only from that person’s own school, classes or work.
- **Access ends when the job ends.** When a teacher leaves a school, they lose access to that school’s classes.
- **The walls are tested.** Automatic tests try to cross these boundaries every time the software changes.

6. Signing In Safely

- **New accounts must confirm their email address** before they can be used.
- **Sessions are short-lived.** A sign-in token is good for one day. While you keep using the site it is renewed quietly in the background, and each renewal replaces the one before. Stop using the site for two days and you must sign in again.
- **Signing out really signs you out.** Signing out or changing a password cancels every session on every device at once.
- **Wrong guesses lock the account.** Five wrong passwords in a row lock an account for fifteen minutes.
- **Reset codes expire quickly.** A password-reset code lasts fifteen minutes, and five wrong codes lock the reset for half an hour.
- **Each new student account gets its own password**, and the student is asked to change it the first time they sign in.
- **Sign in with Google is checked by us**, not taken on trust. Google’s signed proof of identity is verified against Google’s published keys, and it must be made out to Grade A Plus and still in date.
- **Passwords must be at least eight characters**, cannot be a commonly used password, and cannot be made of the person’s own name or email address.

Limits that stop automated guessing

The backend caps how often the same visitor can try a sensitive action. Without a cap, a program could keep trying thousands of times until something worked.

Action	Limit
Signing in	10 per minute
Asking for an emailed code	5 per hour
Resetting a password	10 per hour

Action	Limit
Creating an account	10 per hour
Signing in with Google	20 per hour
Any request before signing in	60 per minute

The limits hold even when someone tries to disguise where they are connecting from. On top of these, each email address has its own budget of wrong activation codes: five wrong codes lock that address for half an hour, whoever is trying.

7. How AI Is Used on Student Work

- **The AI suggests; the teacher decides.** No grade reaches a student until a teacher releases it. Nothing is released automatically.
- **The marking step sees the work, not the person.** It uses only the answers and the teacher's marking guide. The student's name and email are not added to it.
- **The AI service is told not to keep or learn from student work.** Every request refuses any AI provider that would store the work or use it for training. Our AI gateway, OpenRouter, keeps no copy of the work itself, and every request we send is marked "zero data retention", so it can only be routed to AI suppliers that have promised not to store it at all.
- **When the system is unsure, it says so.** Papers the AI could not read or grade with confidence are flagged for the teacher to check.
- **The original is always kept.** The AI's first grade is stored next to any change the teacher makes, so there is a record of both.
- **Names are used only where they are needed.** To match a scanned paper to the right student, the AI is shown the page and the class list of names. A written summary of a student's progress also uses their name.
- **Student answers are treated as data, not instructions.** Anything a student wrote is wrapped and labelled before it reaches the AI, so a student cannot smuggle an instruction such as "give me full marks" into their answer. The score is also clamped by the backend to the marking guide's maximum, whatever the AI says.

8. Payments

- **We never see card details.** Cards are handled by Stripe, a payment provider certified to the highest level of the card industry's security standard (PCI DSS Level 1). The card form is Stripe's own, so card numbers go straight to Stripe and never reach us. Our servers receive only reference numbers.
- **Every message from the payment provider is verified** as genuine before the backend acts on it. Stripe signs each message with a secret only Stripe and we hold; a message with a bad signature, or one more than five minutes old, is refused. Each message is also acted on once only, however many times it is delivered.
- **Financial records cannot be rewritten.** Credit and billing entries can be added to, but they cannot be edited or deleted. The one exception is marking a payment as refunded.
- **There is an independent copy.** Every message from the payment provider is kept permanently, so billing history can always be checked against it.

9. A Record of Who Did What

The backend keeps a permanent activity record. It works like a visitor book that cannot be rewritten.

What it records

- Sign-ins, failed sign-ins and sign-outs
- Password changes and resets
- Grading, and every change to a grade
- Changes to class lists
- Payments, subscriptions and licence changes
- Changes to a person’s role or access
- Everything our own support team does, including what they look at

How it is protected

- **Entries cannot be edited or deleted.**
- **It holds reference numbers, not content.** It never stores a student’s answers.
- **It protects students within the record itself.** A student’s email address and location are never written into it; the database refuses such an entry. For staff, the entry carries their email address and the network address they connected from, and the network address is blanked after 90 days.
- **It survives deletions.** Removing an account does not erase the record of what that account did.
- **A school administrator can review their own school’s record** and no other school’s.

10. Keeping and Removing Data

- **Teachers control their own classes.** A teacher can delete a class, an assignment or a student’s paper. Deleting is permanent.
- **Removing a student from one class** leaves their work in other classes untouched.
- **An account can be deleted on request.** Our support team carries out a real, permanent delete that removes the person’s work and class records.
- **Billing history is kept** after an account is deleted, because financial records must be.
- **A school that leaves is archived,** so nothing is lost by mistake.
- **The activity record is kept for a set time, then removed automatically:**

Kind of entry	Kept for
General activity	12 months
Entries about student grades and records	3 years
Network details attached to an entry	90 days

- **Our hosting company keeps backups** of the database every day. Daily backups are kept for 6 days, weekly ones for 27 days and monthly ones for 89 days. They can only be restored into our own service.

11. Encryption, Layer by Layer

“Encrypted” means scrambled so that only someone with the right key can read it. Data needs protecting in two states: **in transit** (while it travels over a network) and **at rest** (while it sits on a disk). The table says, for each place data goes, what standard protects it and who is responsible for enforcing it.

A few terms, once:

- **TLS** is the standard that puts the padlock in a browser's address bar. TLS 1.2 and 1.3 are the current versions; older ones are considered unsafe.
- **AES-256** is the standard way of scrambling stored data; it is the one banks and governments use.
- **Hashing** is a one-way scramble used for passwords: the password can be checked but never read back.

In transit

Where data travels	How it is protected	Standard	Who enforces it
Between a user's browser and the backend	Every connection uses HTTPS. A plain HTTP request is redirected to HTTPS, and browsers are told to use HTTPS only for the next 30 days.	TLS; certificates issued and renewed automatically by Let's Encrypt	Railway (certificates and the edge), with the redirect enforced by the backend itself
Between the backend and its database and cache, inside the hosting platform	Railway's internal network between services is encrypted end to end	WireGuard (a modern encrypted-tunnel protocol)	Railway
Between the backend and Stripe	HTTPS only; Stripe refuses anything below TLS 1.2	TLS 1.2 or 1.3	Stripe and the backend
Between the backend and the AI gateway (OpenRouter)	HTTPS only	TLS	OpenRouter and the backend
Between the backend and the email services	HTTPS for the API; the mail relay accepts TLS 1.2 and 1.3 only	TLS 1.2 or 1.3	MailerSend, MailerLite and the backend
Between the backend and Google (sign-in)	HTTPS only; Google's reply is a signed token that we verify against Google's published keys	TLS; RS256 signature	Google and the backend
Between the backend and the error-reporting service	HTTPS only; Sentry requires TLS 1.2 or better	TLS 1.2 or 1.3	Sentry
Source code going to and from GitHub	HTTPS or SSH, both encrypted	TLS / SSH	GitHub

The backend never turns certificate checking off for any of these connections.

At rest

What is stored	How it is protected	Standard	Who enforces it
The database (accounts, classes, answers, grades, the activity record)	Encrypted on the disk by the hosting platform; automatic, no key handling on our side	AES-256 (Railway's statement; Railway's contract with us commits to "databases are encrypted at rest")	Railway
Database backups	Encrypted the same way as the database	AES-256 (Railway's statement)	Railway

What is stored	How it is protected	Standard	Who enforces it
Passwords	Never stored. A one-way hash is stored instead, strengthened by being run 1,000,000 times, as recommended by the US standards body NIST	PBKDF2 with SHA-256, Django's default	The backend (Django 5.2)
The keys that let a user sign in with Google	Encrypted a second time by the backend before they are written to the database, with a key held outside the code. If that key is missing the backend refuses to start	Fernet (AES-128 with an HMAC-SHA256 integrity check)	The backend
Card numbers	Never reach us. Stripe stores them in its own vault	AES-256	Stripe
Profile pictures	Stored by Cloudinary, encrypted on disk	AES-256 (Cloudinary's statement)	Cloudinary
Error reports	Stored by Sentry, encrypted on disk, in the United States	Sentry's statement; SOC 2 and ISO 27001 audited	Sentry
Source code	Encrypted on disk by GitHub	GitHub's statement	GitHub
Uploaded papers	Kept privately by Cloudinary and encrypted, for up to 30 days, so we can try again if something goes wrong. Deleted as soon as the paper is processed.	AES-256 (Cloudinary's statement)	Cloudinary

When a teacher or student uploads answers or an assignment, we keep the file privately with our storage provider, Cloudinary, for up to 30 days. This lets us try again if something goes wrong, without asking anyone to upload it again. We delete the file as soon as it has been processed, or after 30 days at the latest.

Where the data lives

The backend and its database run in Railway's data centres in the United States. Error reports are stored in the United States. Email is sent from the European Union, where our email providers are based. The AI suppliers that mark the work are reached through OpenRouter, which is a United States company; the work is sent for marking and, under our zero-retention setting, not kept.

12. Defending the System

An attacker has a short list of tried and tested ways in. This section goes through that list and says what stands in the way of each. The names in brackets are the ones security professionals use, so a school's IT staff can match them to their own checklists.

Against attacks from outside

The attack, in plain terms	What stops it
Guessing a password by trying thousands (brute force, credential stuffing)	Five wrong tries lock the account for 15 minutes; sign-in attempts are capped at 10 per minute per visitor; passwords are checked against a list of common ones
Guessing an emailed code	Codes last 15 minutes; five wrong codes lock the reset for 30 minutes; the address's own budget locks it for 30 minutes whoever is trying
Stealing a sign-in token and using it later (session hijacking)	Tokens expire after a day; signing out or changing the password cancels every token at once; tokens only travel over HTTPS
Tricking the database with specially written text (SQL injection)	The backend never builds a database command out of text a user typed; every query is built by the framework with the data kept separate from the command
Planting a hidden script in something a teacher or student will later view (cross-site scripting)	Everything shown on a page is escaped; text the AI produces is passed through an allow-list that keeps only harmless formatting and drops scripts whole
Making a signed-in user's browser do something they did not ask for (cross-site request forgery)	Only our own websites may send requests with a user's session; the browser's statement of where a request came from is checked on every change
Showing our pages inside a fake page to capture clicks (clickjacking)	Every response tells the browser our pages may not be shown inside another site's frame
Making the backend fetch a private internal address on an attacker's behalf (server-side request forgery)	When the AI assistant is allowed to fetch a web page, the backend refuses private, local and reserved addresses, follows at most five redirects and checks each one, and reads at most 2 MB
Sending a file that is not what it claims to be, or one built to exhaust memory (malicious upload, decompression bomb)	Files are capped at 50 MB; a PDF must really be a PDF whatever its name; at most 300 pages are read; an image may hold at most 50 million pixels
Hiding instructions in a student's answer so the AI gives a better grade (prompt injection)	Student text is wrapped and labelled as data before the AI sees it; the backend clamps every score to the marking guide's maximum; a teacher's custom marking instructions can be switched off centrally
Sending a fake "payment received" message pretending to be Stripe (webhook forgery, replay)	Every message must carry Stripe's signature (HMAC-SHA256) and be less than five minutes old; each message is acted on once only, however many times it arrives
Pointing a request at the wrong site name to confuse the backend (host-header attack)	The backend only answers to its own fixed list of site names
Flooding the backend with requests (denial of service)	Per-visitor rate limits; each request is cut off after 100 seconds; the browser-rendering engine for PDFs has a fixed capacity and sheds extra work with a polite "try again in 5 seconds"
Reading error pages for clues about the inside of the system (information leakage)	Detailed error pages are switched off on the live service; a fault returns a short, uniform message

Against our own mistakes

The mistake	What catches it
A password or key saved into the code by accident	Automatic checks run before every save and refuse a known key pattern or a private key file; keys live only in the hosting environment
Starting the service with a key missing	The backend refuses to start
Personal details ending up in logs or error reports	Email addresses, passwords and tokens are blanked from logs automatically; error reports are sent with personal data, local variables and request bodies switched off, and are scrubbed again on arrival
A change that breaks a security rule	More than 8,000 automatic tests run on every change, including tests that try to cross every access boundary; a change that fails them cannot be merged
A software dependency with a known flaw	Every dependency is pinned to an exact version, so nothing changes without a deliberate update; the code is scanned for unsafe patterns before every save
Running the service with more power than it needs	The backend runs as an ordinary user inside its container, not as the administrator
A change to the database that could lose data	A checker reads every proposed database change and refuses the risky kinds

Known limits, stated plainly

- We rely on our hosting company’s edge for protection against large-scale flooding attacks; the backend’s own limits are per visitor.
- The “cannot be rewritten” rule for financial and activity records is enforced by the application. A database-level lock is being added as a second layer.
- Automated scanning of our dependencies for newly published flaws is being added; today that check is done by hand at each release.
- Inside the hosting platform, the link between the backend and its database is protected by Railway’s encrypted private network. A second, database-level encryption requirement on that link is being added.
- Student answers and feedback inside the database are protected by the platform’s disk encryption (section 11). A second layer, where the backend itself encrypts those fields before they are written, is planned as the next step; it reuses the method already used for the Google sign-in keys.

13. The Laws and Standards We Work To

Grade A Plus is built for schools in the United States. This section names the laws that apply to student data there, the industry standards we are measured against, and the certifications our suppliers hold. Each line says in plain terms what the standard asks for and what we do about it.

A word on wording. Grade A Plus has not itself been audited for a certificate such as SOC 2 or ISO 27001. Where this section says we are *aligned with* a standard, it means our practices follow what it asks for; it does not mean an outside auditor has certified that. Our suppliers’ certifications are theirs, and are listed as such.

Laws that apply to student data

Law	What it asks for, in plain terms	What Grade A Plus does
FERPA (the US federal student-records law, 20 U.S.C. §1232g; 34 CFR Part 99)	A school may share student records with a vendor only as a "school official": the vendor does a job the school would otherwise do itself, stays under the school's direct control for how the records are used, uses them only for that job, and never passes them on	Grade A Plus acts under the school's instructions and only for grading and class management. Student data is never sold, shared or used for any other purpose. Teachers and schools control deletion (section 10) and can see the record of every access (section 9)
COPPA (the US federal law on children under 13, 15 U.S.C. §6501; 16 CFR Part 312, as amended in 2025)	An online service collecting a young child's personal information needs parental consent, which a school may give for educational use; it must collect as little as possible, keep it no longer than needed, keep a written security programme and a written retention policy	We collect only a name and the work (section 4): no birth date, no address, no photo of the child. We keep student data only for the school's purpose and delete it on request. Our retention periods are written in section 10. This document and the engineering team's backend reference (`docs/backend/`) are our written security programme
State student-privacy laws (for example California's SOPIPA and AB 1584, New York's Education Law §2-d, Illinois' SOPPA)	No targeted advertising, no profiling, no selling of student data; reasonable security; deletion on the school's request; a contract that keeps the records the school's property; encryption in transit and at rest; notice to the school of any breach	Grade A Plus carries no advertising and builds no profiles. Records stay the school's. Encryption is set out in section 11. We sign each school's own data agreement where state law requires one, and we notify the school of any breach affecting its data

Industry standards we are aligned with

Standard	What it covers, in plain terms	How we use it
PCI DSS (the card industry's security standard, version 4.0.1)	Anyone taking card payments must protect card data. Using Stripe's hosted card form puts us in the lightest category (SAQ A), because card data never touches our systems	Section 8. Stripe is certified at the highest level (Service Provider Level 1)
NIST SP 800-63B (the US standards body's guide to passwords and sign-in)	Lock accounts after repeated wrong guesses; check passwords against lists of common ones; do not force pointless password rotation; make emailed codes short-lived	Section 6 follows it: lockouts, a common-password check, no forced rotation, 15-minute codes
NIST Cybersecurity Framework 2.0 (the US framework for running security as a programme: govern, identify, protect, detect, respond, recover; New York's Education Law §2-d requires vendors to align with it)	A structured way to manage security risk, not a checklist of technology	We keep a written list of weaknesses found and fix them in order of seriousness (section 15), keep a permanent activity record (section 9), and report faults automatically (section 3)
OWASP Top 10 and ASVS (the web-security community's list of the ten most common flaws, and its detailed verification standard)	Access control, cryptography, injection, insecure design, misconfiguration, outdated components, authentication, data integrity, logging, server-side request forgery	Section 12 is organised around this list; every line has a defence and an automatic test

Standard	What it covers, in plain terms	How we use it
Student Privacy Pledge (a voluntary public promise by education-technology companies, enforceable by the Federal Trade Commission once signed)	No selling of student data, no behavioural advertising, no profiling outside the educational purpose, security, deletion, transparency	Our practice already matches every commitment; signing the Pledge is a decision for the company's leadership

What our suppliers are certified to

Supplier	Certification or attestation	What it means
Railway (hosting, database)	SOC 2 Type II and SOC 3	An independent accountant has examined Railway's security controls over a period of months and reported on them
Stripe (payments)	PCI DSS Service Provider Level 1; SOC 1 and SOC 2 Type II	The card industry's highest level; audited every year
OpenRouter (AI gateway)	SOC 2 Type 2	Independently examined security controls; penetration-tested
Sentry (error reports)	SOC 2 Type II; ISO 27001	Independently examined and certified to the international information-security standard
Cloudinary (profile pictures, uploaded papers)	SOC 2 Type II; ISO 27001 family	As above
MailerLite (email)	ISO/IEC 27001:2022	Certified to the international information-security standard; hosted in the EU
MailerSend (email)	Hosted in an ISO 27001-certified data centre in the EU	The certificate is the data centre's
GitHub (source code)	SOC 2; ISO 27001	Independently examined and certified
Google (sign-in)	OpenID Certified; Google Cloud holds ISO 27001 and SOC 2	Google's sign-in follows the open standard exactly; its infrastructure is certified

14. The Companies We Work With

We use a small number of well-known companies. Each receives only what it needs for its job. We do not sell data to anyone.

Job	Company	What it receives	Where
AI marking	OpenRouter	The work to be marked and the marking guide, marked "do not keep"	United States
Payments	Stripe	The payer's name and email, and the card details they type into Stripe's own form	United States
Sending email	MailerSend, MailerLite	Email addresses and the messages we send	European Union
Sign in with Google	Google	Only what is needed to confirm who is signing in	United States

Job	Company	What it receives	Where
Profile pictures	Cloudinary	Pictures users choose to upload	United States
Uploaded papers	Cloudinary	Answer and assignment files that teachers and students upload, kept until processed or for at most 30 days	United States
Error reports	Sentry	Technical details of faults, with personal details left out	United States
Hosting	Railway	Runs the backend and its database	United States
Source code	GitHub	The backend’s code, never any school data	United States

15. How We Check Our Own Work

Security claims are only as good as the checking behind them. This section says how the backend is tested, in enough detail that a school’s IT staff can judge it.

The automatic tests

- **More than 8,000 automatic tests** cover the whole backend (8,376 at the time of writing). Each one sets up a small situation, asks the backend to act, and checks the answer.
- **They run on every change.** Each change pushed to the code repository is tested automatically by GitHub before it can be merged, together with a check that any database change is safe to apply.
- **They run in full before every release**, on the team’s own machine, under a strict runner that records the whole log, stops a run that stalls or runs out of memory, and keeps a checksummed copy of the log as evidence.
- **No test touches the outside world.** During tests, a guard refuses any real connection to the internet: the AI, Stripe, Google and the email services are all stood in for. A test that tried to reach a real service would fail.

What the tests prove, by kind:

Kind of test	What it proves
Boundary tests	A teacher cannot see another teacher’s class; a school administrator cannot see another school; a student cannot see a grade before it is released; a signed-out visitor gets nothing
Sign-in tests	Lockouts, code expiry, the rate limits in section 6, “sign out everywhere”, and the checks on Google’s signed proof of identity all behave as stated
Payment tests	A message without Stripe’s signature is refused; a message delivered twice is acted on once; credits are counted correctly and refunded when AI work fails
AI pipeline tests	Papers are matched to the right student, scores are clamped to the marking guide, a student’s text is treated as data, and the “do not keep” instruction is on every request

Kind of test	What it proves
Upload tests	Oversized files, files that are not really PDFs, and image bombs are refused
Privacy tests	No email address reaches a log; a student’s entry in the activity record carries no email or network address; error reports are scrubbed
Rendering tests	The printable PDF of an assignment is produced by a real browser engine, under its capacity limit
Database-change tests	A proposed change to the database layout that could lose data or lock a table is caught before it is applied

How a fix is made

Every security fix follows the same written procedure, and the record of each one is kept.

1. **The test comes first, and it must fail.** Before anything is changed, a test is written that shows the problem. It is run and its failure is written down, word for word, before the fix. If it does not fail as predicted, the problem was not understood.
2. **The fix is made, and the test passes.**
3. **We test the test.** The fixed code is deliberately broken in several ways (each a “mutant”), and the tests must catch every one. A mutant that survives means a test is too weak. There are 94 such exercises on record.
4. **The change is checked for spread.** An automatic check confirms the change touched only the files it was meant to and nothing else.
5. **A second person checks.** An independent reviewer re-reads and re-runs the work and records a verdict; 219 such verification records are on file. The person who writes a change is never the person who approves it.
6. **The evidence is kept.** The logs, the mutants and the verdicts for each fix are stored in the code repository (266 evidence folders at the time of writing), so any claim in this document can be traced back to the run that proved it.

Checks before every save

Twenty-six automatic checks run on the engineer’s machine before any change can be saved: code formatting, type checking, a scan for unsafe code patterns, a scan for anything that looks like a password or key, a check for private-key files, a check that no personal detail is passed to a log, a limit on file size, and a rule that a timeout in one place stays in step with its partner in another.

Checks against the real world

Ordinary tests check what the team believes the outside services do. Two harnesses check the belief against the service itself:

- **AI accuracy.** A set of papers marked by hand is kept as a benchmark. Every night the AI’s past answers are replayed against it (at no cost), and every week the live AI marks the set again. The scores are tracked over time, so a quiet drop in accuracy — whether from our change or the AI company’s — is noticed.
- **Payments.** A separate harness exercises Stripe’s test environment with real Stripe messages, including “time travel” to simulate a renewal months ahead. It refuses to run against anything but a test key, so it can never touch a real payment.

Looking for our own weaknesses

- **We run regular security reviews** and keep a written list of what we find, fixing the items in order of seriousness. The list currently runs to more than 250 items, most of them closed.
- **We attack ourselves.** Red-team exercises have tried to take over an account through the Google sign-in, to reuse a cancelled sign-in token, to change another user's password, and to read another school's students. Each exercise's method and result is on file, and the fixes it led to are in the list above.
- **Changes are rehearsed first.** Every release is proven on a practice copy of the service before it reaches users.
- **Nothing reaches users without the founder's sign-off.**

16. At a Glance

Area	How the backend handles it
Encrypted connections	Required and enforced (HTTPS only; TLS)
Encryption at rest	Database and backups encrypted by the hosting platform (AES-256); passwords hashed; Google keys encrypted a second time
Sign-in for every request	Enforced
Role-based access control	Decided by the backend on every request
Separation between schools	Enforced and tested
Password storage	Hashed, never readable (PBKDF2-SHA256, 1,000,000 rounds)
Account lock after wrong passwords	Implemented
Limits on repeated attempts	Implemented
Session expiry and sign-out everywhere	Implemented
Student grades released only by a teacher	Enforced
AI service barred from keeping student work	Implemented (zero data retention on every request)
Card details	Never stored; Stripe, PCI DSS Level 1
Financial records	Cannot be edited
Audit logging	Permanent activity record
Secrets and keys	Kept outside the code, verified
Personal details in logs and error reports	Left out
Account deletion	Permanent, on request
Laws we work to	FERPA, COPPA, state student-privacy laws
Standards we are aligned with	PCI DSS (SAQ A), NIST 800-63B, NIST CSF 2.0, OWASP Top 10 / ASVS
Testing	8,376 automatic tests on every change; test-first fixes; mutation-tested; independently verified; evidence kept
Certifications held by Grade A Plus	None yet; suppliers' certifications are listed in section 13

17. Reporting a Security Concern

If you discover a security or privacy issue in Grade A Plus, please report it to the project maintainers privately. Please do not post details of an unpatched vulnerability publicly.

Appendix: Where Each Statement Comes From

This appendix is for a school's IT or legal staff who want to check a statement. Each line gives the source: either the part of the backend's code that enforces it (read on 26 October 2026) or the supplier's or regulator's own published page.

Our own code

Statement	Where it is enforced
HTTPS redirect, 30-day HTTPS-only instruction to browsers, secure cookies	`AutoGrader/settings.py` lines 337-356 (`SECURE_SSL_REDIRECT`, `SECURE_HSTS_SECONDS`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`); the redirect switch is set on the live service
Only our own websites	`AutoGrader/settings.py` lines 275-327 (`CORS_ALLOWED_ORIGINS`, `CORS_ALLOW_CREDENTIALS = False`)
Fixed list of site names	`AutoGrader/settings.py` lines 250-268 (`ALLOWED_HOSTS`)
Password hashing, 1,000,000 rounds	Django 5.2.6 default (`PBKDF2PasswordHasher`); password rules at `settings.py` lines 507-520
Google keys encrypted a second time	`users/models.py` lines 374-382 (`EncryptedCharField`, django-encrypted-model-fields on the `cryptography` library's Fernet); key at `settings.py` line 1463
Sign-in tokens: one day, renewal, cancel-everywhere	`settings.py` lines 1284-1288; `users/models.py` lines 147-154 and 297-339 (`token_epoch`); `users/authentication.py` lines 51-62
Account lockout 5 / 15 minutes	`users/models.py` lines 93-94 and 343-361
Reset code 15 minutes; 5 wrong / 30 minutes	`users/models.py` lines 440-441 and 459-480
Per-address code budget	`users/throttling.py` lines 140-240; `settings.py` lines 1279-1280
Rate limits table	`settings.py` lines 1237-1244; `users/throttling.py` lines 26-65
Google sign-in verification	`users/views.py` lines 2222-2230 (`google-auth` `verify_oauth2_token`: signature, expiry, issuer, audience)
AI zero data retention	`ai_processor/services.py` lines 744-779 (`data_collection: "deny"`, `zdr: true`); switch `AI_PROVIDER_ZDR_ENABLED`, on for the live service
Student answers treated as data; score clamp	`ai_processor/services.py` lines 803-913 and 3060
Stripe signature, five-minute window, once-only handling	`billing/webhooks.py` lines 416-468 and 131-170; `billing/models.py` lines 2418-2445
Financial and activity records cannot be rewritten	`billing/immutable.py`; `billing/models.py` lines 1336, 1525, 1683; `audit/models.py` line 12

Statement	Where it is enforced
Student email and location never in the activity record	`audit/emitter.py` lines 515-531; `audit/models.py` lines 93-105 (database constraint `audit_student_no_pii_ck`)
Activity record retention 12 months / 3 years / 90 days	`audit/tasks.py` lines 25-94; `settings.py` lines 1078-1088
Upload caps: 50 MB, real PDF, 300 pages, 50 million pixels	`AutoGrader/uploads.py` line 37; `ai_processor/services.py` lines 5999 and 6048-6071; `assignments/services.py` line 208
Uploaded file kept until success or 30 days	`students/kept_files.py`, `students/kept_uploads.py`, `students/tasks.py` (purge)
Scripts stripped from AI text	`assignments/services.py` lines 657-680 (`bleach` allow-list)
Fetching web pages safely (private addresses refused, 5 redirects, 2 MB)	`ai_processor/tools.py` lines 27-208
No database command built from typed text	Django ORM throughout; no `.raw()`, `.extra()` or `RawSQL` in application code
Frame protection, content-type protection, referrer policy	Django `SecurityMiddleware` and `XFrameOptionsMiddleware` defaults (`settings.py` lines 399 and 407)
Keys outside the code; refuse to start if missing	`settings.py` line 115 and throughout (`env.str` with no default)
Secrets caught before saving	`.pre-commit-config.yaml`: `detect-secrets` v1.5.0, `detect-private-key`, `check-no-pii-in-logs`
Logs scrubbed	`AutoGrader/log_scrubbing.py` lines 102-240
Error reports scrubbed	`settings.py` lines 217-236 (`send_default_pii = False`, `include_local_variables = False`, `max_request_body_size = "never"`); `AutoGrader/sentry_scrubbing.py`
Detailed error pages off	`settings.py` lines 133-151 (`DEBUG = False` on the live service)
Runs as an ordinary user	`Dockerfile` lines 5 and 65
100-second request cut-off; PDF renderer capacity	`Dockerfile` line 86; `docs/backend/pdf-pipeline.md`
Dependencies pinned	`requirements.txt` (every line `==`)
Database-change checker	`scripts/check_migration_safety.py`; `.github/workflows/migration-safety.yml`
Tests on every change	`.github/workflows/tests.yml` and `pre-commit.yml`; 8,376 tests on 26 October 2026 (`docs/evidence/beta-batch-28/`)
No real outside connection during tests	`AutoGrader/network_guard.py`
Test-first fixes, mutants, verification, evidence	`docs/evidence/` (266 folders; 94 mutation harnesses; 219 verification records); the procedure in `docs/CODE_REVIEW_STANDARDS.md`
Twenty-six checks before every save	`.pre-commit-config.yaml`; the table of a run at `docs/evidence/beta-batch-28/hook_table_6db6c4b6.txt`
AI accuracy benchmark; payments harness	`docs/backend/ai-quality-harness.md`; `docs/backend/billing-qa-harness.md`

Statement	Where it is enforced
Red-team exercises	`docs/evidence/authz-oauth-takeover/`, `authz-token-epoch/`, `authz-patch-password/`, `my_students_tenancy/`

Suppliers and regulators

Statement	Source
Railway: databases encrypted at rest; data in transit encrypted with WireGuard; daily backups	Railway Data Processing Addendum, Exhibit C — https://railway.com/legal/dpa
Railway: AES-256 for data at rest including volumes and backups	Railway staff statement, 17 August 2026 — https://station.railway.com/questions/data-protection-questions-encryption-a-e5128546
Railway: SOC 2 Type II and SOC 3	https://docs.railway.com/enterprise/compliance ; https://trust.railway.com/
Railway: certificates from Let’s Encrypt, renewed automatically	https://docs.railway.com/networking/troubleshooting/ssl
Railway: backup schedule	https://docs.railway.com/reference/backups
Railway: US regions	https://docs.railway.com/reference/regions
Stripe: PCI Service Provider Level 1; SOC 1 and 2; TLS 1.2 minimum; card vault AES-256	https://docs.stripe.com/security
Stripe: hosted card form and SAQ A	https://stripe.com/guides/pci-compliance
Stripe: webhook signatures (HMAC-SHA256, five-minute tolerance)	https://docs.stripe.com/webhooks
OpenRouter: keeps no prompts; does not train on inputs	https://openrouter.ai/docs/features/zdr ; https://openrouter.ai/privacy
OpenRouter: zero-data-retention routing and `data_collection: deny`	https://openrouter.ai/docs/features/provider-routing ; https://openrouter.ai/docs/features/zdr
OpenRouter: SOC 2 Type 2	https://trust.openrouter.ai/
MailerSend: EU hosting, TLS 1.2/1.3 relay	https://www.mailersend.com/legal/security-statement ; https://www.mailersend.com/help/smtp-relay
MailerLite: ISO/IEC 27001:2022, EU hosting	https://www.mailerlite.com/legal/security-statement
Google: how a sign-in token must be verified; OpenID Certified	https://developers.google.com/identity/gsi/web/guides/verify-google-id-token ; https://developers.google.com/identity/openid-connect/openid-connect
Cloudinary: SOC 2; ISO 27001 family; AES-256 at rest; TLS 1.2+	https://cloudinary.com/trust ; https://cloudinary.com/trust_03_31_2023
Sentry: SOC 2 Type II, ISO 27001, encryption at rest and in transit, TLS 1.2 minimum, US data region, data scrubbing	https://sentry.io/trust/ ; https://sentry.io/legal/dpa/ ; https://docs.sentry.io/security-legal-pii/security/ssl/ ; https://docs.sentry.io/security-legal-pii/scrubbing/server-side-scrubbing/
GitHub: encrypted at rest; SOC 2 and ISO	https://github.blog/changelog/2019-05-23-git-data-encryption-at-rest/ ; https://github.com/security
PostgreSQL: encryption at rest is the host’s job, not the database’s	https://www.postgresql.org/docs/current/encryption-options.html
Django: PBKDF2-SHA256 default, 1,000,000 rounds in 5.2; CSRF; SQL injection; security headers	https://docs.djangoproject.com/en/5.2/topics/auth/passwords/ ; https://docs.djangoproject.com/en/5.2/releases/5.2/ ; https://docs.djangoproject.com/en/5.2/topics/security/

Statement	Source
FERPA: the “school official” conditions and no re-disclosure	34 CFR §99.31(a)(1)(i)(B) and §99.33 — https://www.law.cornell.edu/cfr/text/34/99.31 ; https://www.law.cornell.edu/cfr/text/34/99.33
COPPA: 2025 amended rule; written security programme (§312.8); written retention policy (§312.10); school consent	90 FR 16918 — https://www.govinfo.gov/content/pkg/FR-2025-04-22/pdf/2025-05904.pdf ; FTC FAQ section N — https://www.ftc.gov/business-guidance/resources/complying-coppa-frequently-asked-questions
California SOPIPA; AB 1584	https://codes.findlaw.com/ca/business-and-professions-code/bpc-sect-22584/ ; https://codes.findlaw.com/ca/education-code/edc-sect-49073-1/
New York Education Law §2-d	https://www.nysenate.gov/legislation/laws/EDN/2-D
Illinois SOPPA	105 ILCS 85 — https://www.ilga.gov/legislation/ilcs/ilcs3.asp?ActID=3806&ChapterID=17
Student Privacy Pledge	https://fpf.org/student-privacy-pledge/
NIST SP 800-63B	https://pages.nist.gov/800-63-4/sp800-63b.html
NIST Cybersecurity Framework 2.0	https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf
OWASP Top 10; ASVS	https://top10.owasp.org/ ; https://asvs.owasp.org/
PCI DSS v4.0.1	https://www.pcisecuritystandards.org/standards/pci-dss/

This document is a plain-language summary intended for a general audience. The detailed technical notes behind it are the engineering team’s backend reference, kept in the code repository under `docs/backend/` (start with `security-and-tenancy.md` and `users-and-auth.md`), and the written list of weaknesses found and fixed, `docs/HARDENING_BACKLOG.md`.